

华东师范大学《编程综合实践》课程

项目报告书

主题	智能选课系统		
姓名	王业涵	学号	10245102458
电子邮箱	yehanw133@gmail.com	指导教师	肖春芸 贺樑

概要设计

一、环境配置

- 1.语言：C++11；
- 2.GUI 框架：QMainWindow；
- 3.构建工具：CMake for Qt 6.5 and later；
- 4.环境：Qt Creator 6.9.0。

二、项目概览

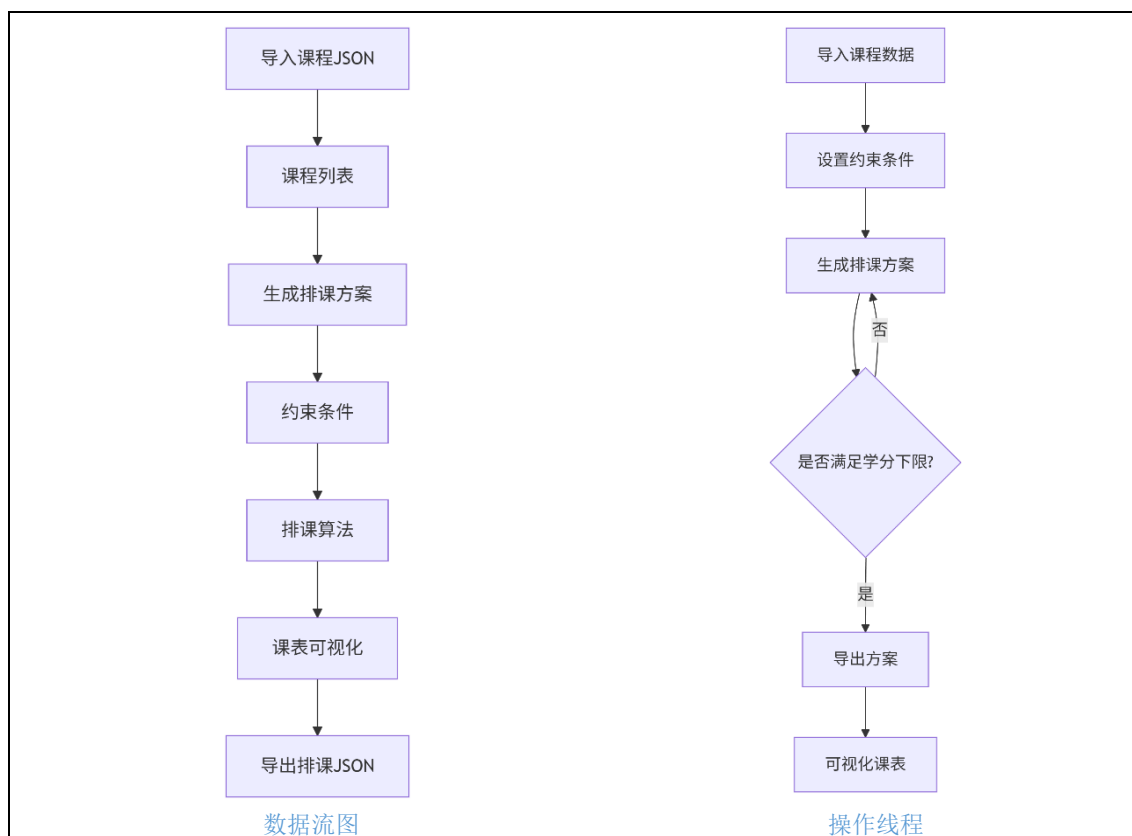
1. 核心数据结构：
 - (1) 培养方案结构 (Course 类)

```
class Course {
public:
    QString id;
    QString name;
    int credit;
    QString semester; // "Autumn" or "Spring"
    QString required; // "Compulsory" or "Elective"
    struct Offering {
        QString id;
        QString teacher;
        QList<int> times; // 7 天，每天 13 节课的位掩码
        int weeks;       // 18 位周次掩码
    };
    QList<Offering> offerings; //表示这门课所有可选教学班列表（检测时间冲突）
    QList<QString> prerequisites; //表示这门课的先修课程 ID 列表（验证先修关系）
};
```

(2) 选课方案存储结构

m_courseSemesterMap: 课程 ID 到学期的映射 (QHash<QString, int>);
 m_courseOfferingMap: 课程 ID 到教学班 ID 的映射 (QHash<QString, QString>);
 m_semesterCredits: 每个学期的总学分 (QList<int>);
 m_forbiddenSlots: 每个学期的禁排时间段 (QList<QList<int>>).

2. 数据流图与操作线程:



3.模块说明:

功能按钮	所属模块	模块功能	技术实现
生成排课	核心算法模块	根据课程约束条件自动生成最优排课方案，同时识别连续课程	贪心算法、位运算
添加课程	核心算法模块	允许用户添加课程，含课程 ID、课程名称、学分、开课学期、课程类型、先修课程和上课时间	QFormLayout、输入验证、JSON 数据解析
编辑课程	核心算法模块	允许用户修改课程信息，含课程名称、学分、开课学期、课程类型和先修课程	QFormLayout、输入验证、JSON 数据解析
删除课程	核心算法模块	从系统中移除指定课程	Qt 表格视图操作、QList 删除操作
连续课程	核心算法模块	允许用户指定两个及以上连续课程（即课程开课学期连续）	拓扑排序、DAG 检测、DFS 循环检测
查找课程	查询处理模块	根据关键词实时筛选课程	字符串匹配、多列搜索、实时过滤
导入课程	文件处理模块	从 JSON 文件批量导入课程数据	JSON 数据解析、文件 IO 操作
导出排课	文件处理模块	将排课结果保存为 JSON 文件	JSON 序列化、文件 IO 操作
交互界面	交互界面模块	管理图形界面，包括课程表格、学期课表选项卡和功能按钮	Qt Widgets 布局管理、Qt 信号槽

详细设计

一、程序结构

SmartCourseScheduler

- |—— mainwindow.h # 主窗口类声明
- |—— mainwindow.cpp # 核心逻辑实现
- |—— main.cpp # 程序入口
- |—— mainwindow.ui # 界面布局文件

二、功能实现

1. 生成排课——**generateSchedule**

```
void MainWindow::generateSchedule(int minCredits) {
    // 1. 重置所有学期数据
    for (int i = 0; i < 8; ++i) {
        m_scheduleModels[i]->clear();    // 清空课表模型
        m_semesterCredits[i] = 0;        // 重置学分计数器
    }
    m_courseSemesterMap.clear();          // 清空课程-学期映射
    m_courseOfferingMap.clear();          // 清空课程-教学班映射
    // 2. 识别连续课程对（如 CS101 和 CS102）
    findConsecutiveCoursePairs();
    // 3. 按学期循环排课
    for (int semester = 0; semester < 8; ++semester) {
        fillSemesterToMaximum(semester, m_courses, minCredits);
        // 4. 处理连续课程（必须在相邻学期）
        for (auto it = m_courseSemesterMap.begin(); it != m_courseSemesterMap.end(); ++it) {
            if (it.value() == semester) {
                scheduleConsecutiveCourse(it.key(), semester);
            }
        }
    }
    // 5. 最终验证
    bool valid = validateScheduleTimeConflicts() && validatePrerequisites();
    displaySchedule(); // 刷新 UI 显示
}
```

2. 添加课程——**on_addCourseButton_clicked**

```
void MainWindow::on_addCourseButton_clicked() {
    /*UI 可视化细节隐藏不表*/
    creditSpin.setRange(1, 10); // 设置学分范围（1-10）
    if (dialog.exec() == QDialog::Accepted) { // 提交时创建新课程对象
        Course newCourse;
        newCourse.id = idEdit.text();
        newCourse.name = nameEdit.text();
        newCourse.credit = creditSpin.value();
        QStringList preReqs = preReqEdit.text().split(','); // 解析先修课程（逗号分隔）
    }
}
```

```

        for (const QString &preReq : preReqs) {
            newCourse.prerequisites.append(preReq.trimmed());
        }
        m_courses.append(newCourse); // 添加到课程列表并刷新显示
        displayCourses();
    }
}

```

3. 编辑课程——**on_editCourseButton_clicked**

```

void MainWindow::on_editCourseButton_clicked() {
    QModelIndex index = ui->courseTableView->currentIndex(); // 获取当前选中课程
    QString courseId = m_courseModel->item(index.row(), 0)->text();
    Course *course = nullptr; // 查找课程对象
    for (Course &c : m_courses) {
        if (c.id == courseId) {
            course = &c;
            break;
        }
    }
    if (dialog.exec() == QDialog::Accepted) { // 更新课程属性
        course->name = nameEdit.text();
        course->credit = creditSpin.value();
        course->prerequisites = preReqEdit.text().split(','); // 更新先修课程列表
        displayCourses(); // 刷新 UI
    }
}

```

4. 删除课程——**on_removeCourseButton_clicked**

```

void MainWindow::on_removeCourseButton_clicked() {
    QString courseId = m_courseModel->item(index.row(), 0)->text(); // 获取选中课程 ID
    for (int i = 0; i < m_courses.size(); ++i) { // 从课程列表中移除
        if (m_courses[i].id == courseId) {
            m_courses.removeAt(i);
            break;
        }
    }
    m_courseSemesterMap.remove(courseId); // 从排课结果中移除（如果已安排）
    m_courseOfferingMap.remove(courseId);
    displayCourses(); // 刷新显示
}

```

5. 查找课程——**on_searchTextChanged**

```

void MainWindow::on_searchTextChanged(const QString &text) {
    for (int row = 0; row < m_courseModel->rowCount(); ++row) { // 实时过滤课程表格
        bool match = false;
        // 多列搜索（ID/名称/学期/类型）
        for (int col = 0; col <= 4; ++col) {

```

```

        QStandardItem *item = m_courseModel->item(row, col);
        if (item && item->text().contains(text, Qt::CaseInsensitive)) {
            match = true;
            break;
        }
    }
    ui->courseTableView->setRowHidden(row, !match);
}
}
}

6.连续课程——addConsecutiveCoursePair
void MainWindow::addConsecutiveCoursePair(const QString &preCourseId, const QString
&postCourseId) { // 添加连续课程对（前导→后续）
    if (!findCourseById(preCourseId) || !findCourseById(postCourseId)) // 检查课程是否存在
        return;
    QSet<QString> visited; //成环检测
    QString current = postCourseId;
    while (!current.isEmpty()) {
        if (visited.contains(current)) {
            qDebug() << "检测到循环依赖:" << preCourseId << "->" << postCourseId;
            return;
        }
        visited.insert(current);
        current = getConsecutiveCourse(current); // 沿着关系链查找
    }
    m_consecutiveCourseMap[preCourseId] = postCourseId; // 存储关系
}
}

```

调试分析

一、重要算法分析

1.生成排课算法：void generateSchedule(int minCredits);

(1)时间复杂度： $O(n^2)$ ，主导项为 **findConsecutiveCoursePairs()** 的双重循环和 **fillSemesterToMaximum()** 中的课程排序和冲突检查；

b.平均情况：接近 $O(n^2)$ ，因为即使提前终止，算法仍需遍历大部分课程。

(2)空间复杂度： $O(n)$ ，m_courseSemesterMap 和 m_courseOfferingMap 分别存储 n 个课程的安排信息。

2.连续课程识别：void findConsecutiveCoursePairs();

```

void findConsecutiveCoursePairs() {
    for (int i = 0; i < m_courses.size(); ++i) { //  $O(n)$ 
        for (int j = i + 1; j < m_courses.size(); ++j) { //  $O(n)$ 
            if (isConsecutiveCoursePair(...)) { //  $O(1)$ 
                m_consecutiveCoursePairs.append(...); //  $O(1)$ 
            }
        }
    }
}
}
}

```

- (1)时间复杂度: $O(n^2)$, 双重循环遍历所有课程对;
 (2)空间复杂度: $O(n)$, 最坏情况下可能有 $n/2$ 对连续课程。
 3.时间冲突检测: bool hasTimeConflictInSemester(...)

```
bool hasTimeConflictInSemester(...) {
    for (auto it = m_courseSemesterMap.begin(); it != ...; ++it) { //  $O(n)$ 
        if (it.value() == semester) {
            for (const auto &offering : ...) { //  $O(m)$ ,  $m$ 为教学班数
                for (int day = 0; day < 7; ++day) { //  $O(1)$ 
                    if (newOffering.times[day] & existing...) //  $O(1)$ 
                        return true;
                }
            }
        }
    }
    return false;
}
```

- (1)时间复杂度: $O(n * m)$, 其中 n 为已排课程数, m 为单个课程的教学班数;
 (2)空间复杂度: $O(1)$, 仅需固定大小的临时变量。

二、困难及解决方案

1.智能排课算法优化与学分控制

问题: 原本算法中的排课算法会导致分数超出上限, 从而不能完成控制单学期学分上限这一功能。

解决: 删除了所有超学分相关函数(scheduleCourseAggressive、scheduleCourseOverCredit等), mainwindow.cpp的代码用2000行减少至1500行, 原本设置这些函数是因为算法不够优秀, 无法生成总学分超过300的选课方案, 故采取激进策略。同时修改了generateSchedule函数, 简化为单轮排课策略, 最后优化了scheduleCourse函数, 确保严格不超出学分上限。这样就从原来的多轮激进策略简化为单轮严格约束策略, 以保证不超出上限的情况下达到总学分最大化。

2.时间冲突检测算法实现

问题: 在代码保证了总学分提升的情况下出现了课程时间冲突问题, 无法通过checker。

解决: 设计算法 bool hasTimeConflictWithOffering(const Course &a, const Course &b, const Course::Offering &offeringA)使用位运算进行时间槽位映射, 支持教学班级别的冲突检测, 实现了7天×13节课的时间矩阵。另外, 添加验证机制 bool validateScheduleTimeConflicts(), 支持全课表时间冲突验证和具体冲突信息输出

3.先修关系验证与连续课程管理

问题: 在优化排课算法以及完善扩展内容连续课程管理的过程中出现没顾及先修关系的情况, 无法通过checker。

解决: 设计先修关系验证算法 bool isPrerequisiteSatisfied(const QString &courseId, int currentSemester), 确保先修课程在当前学期之前完成。识别连续课程(如数学分析I和数学分析II)是本项目的一大难点, 经过王业涵同学的分析, 他开创性地总结出了连续课程的充要条件:

给定的若干个课程是连续课程

iff

(1)课程代码（ID）的数字部分差的绝对值等于 1。如数学分析I的 ID 为 COMS0031121013，数学分析II的 id 为 COMS0031121014；

(2)这两门课所有的先修课程中，一定有一门包含这两门课中的一门。如数学分析I没有先修课程，数学分析II的先修课程是 COMS0031121013（即数学分析I）；

(3)这两门课的开课学期不能同时是 Spring 或者 Autumn。如数学分析I开课在 Autumn，数学分析II在 Spring 开课。

这样就可以判定连续课程了。事实上，在给定的 course.json 文件中，只有数学分析I、数学分析II和大学物理 B（一）、大学物理 B（二）是连续课程。

然而，这样巧妙的思路似乎不太符合命题立意，为了符合“自行修改数据”的需求，开发者请求 cursor 制作新的交互功能，允许用户显示指定连续课程，结果见 mainwindow.cpp 中的 void MainWindow::addConsecutiveCoursePair(const QString &preCourseId, const QString &postCourseId)函数。

用户手册

一、使用指南

0.在正式使用此产品前，用户需确保已安装 Qt Creator 6.9.0 或更高版本，如果没有安装，可参照下面指示自行安装 Qt Creator 6.9.0。

(1)软件下载

a.使用浏览器（建议选择 Google Chrome 或 Edge）前往 <https://www.qt.io/>，单击右上角 "Download. Try."；

b.页面右侧出现"Community User"一栏，单击"Explore Qt Community Edition"；

c.下拉页面直至出现"Get Started"字样，单击其上方的"Download the Qt Online Installer"；

d.根据操作系统选择合适的版本，下载即可。

(2)账号注册

a.在上一步的界面（<https://www.qt.io/download-qt-installer-oss>）单击用户图标；

b.单击"Create Qt Account"，创建一个账号；

c.输入邮箱地址并创建密码，验证密码，输入验证码，勾选"I accept the Qt Account terms."，单击"Create Qt Account"；

d.当页面显示"Your Qt Account has been created."后，在刚刚填写的邮箱中找到 Qt Creator 自动发送的邮件，点击邮件中的链接以激活账号。

(3)本地安装

a.双击打开刚才下载的.exe 安装器，登录账户，点击“下一步”；

b.勾选“我已阅读并同意使用开源 Qt 的条款和条件”和“我是个人用户，我不为任何公司使用 Qt”，请勿在方框中输入任何内容，否则将无法免费使用；

c.等待程序从远程仓库检索信息，检索完毕后页面将自动跳转，用户自行选择"Help us to improve by enabling sending pseudonymous usage statistics in Qt Creator"或"Disable sending pseudonymous usage statistics in Qt Creator"；

d.指定安装目录（不建议选择 C 盘），特别注意，安装路径中不能出现中文，这可能会导致 Qt Creator 无法正常运行；

e.勾选安装组件，确保选择的磁盘中有足够的空间来安装 Qt Creator。

(4)项目导入

a.双击打开 Qt Creator，依次单击左上角“文件(F)”、“打开文件或项目(O)”；

b.选择文件夹 SmartCourseScheduler 中的"CMakeList.txt"，单击打开；

c.双击屏幕左侧的具有文件夹图标的"SmartKeyLab"以展开，双击具有榔头图标的"SmartCourseScheduler"再次展开，依次双击"Header Files"和"Source Files"，可以看到如下界

面:

SmartCourseScheduler

- ├── CMakeList.txt
- ├── Header Files
 - └── mainwindow.h
- ├── Source Files
 - └── main.cpp
 - └── mainwindow.cpp
- └── Forms
 - └── mainwindow.ui

(4)单击界面左侧的绿色三角形，即可开始使用；

(5)欲查看代码内容，单击"mainwindow.h"、"main.cpp"、"mainwindow.cpp"即可。

1.导入课程：单击“导入课程”即可导入培养方案。

The screenshot shows the SmartCourseScheduler application window. The title bar is '智能选课系统'. The menu bar includes '文件' (File) and '帮助' (Help). The toolbar contains buttons for '导入课程' (Import Course), '生成排课' (Generate Scheduling), '导出排课' (Export Scheduling), '添加课程' (Add Course), '编辑课程' (Edit Course), '删除课程' (Delete Course), '设置约束' (Set Constraints), '连续课程' (Continuous Course), '总学分: 0' (Total Credits: 0), and '搜索课程...' (Search Course...). The main area is divided into two tabs: '课程列表' (Course List) and '排课方案' (Scheduling Scheme). The '课程列表' tab is active, displaying a table of courses. Annotations include: a blue circle around the toolbar with text '核心功能区, 用户根据需要选择操作'; a yellow box around '选课方案 总学分统计'; a green box around the search button with text '输入课程id或课程名称以检索课程'; and a thought bubble pointing to the '导入课程' button with text '导入 course.json 后的课程列表'.

	ID	名称	学分	学期	类型	先修课
1	COEN0031112043	体育赛事英语	4	Spring	Elective	COEN0031111109
2	COEN0031111003	英文歌曲文化赏析	4	Autumn	Elective	COEN0031111109
3	SHKX0031111008	毛泽东思想和中国特色社会主义理论体系概论	6	Spring	Compulsory	SHKX0031111017, SHKX0031111018
4	SHKX0031111016	思想道德与法治	6	Autumn	Compulsory	
5	COMS0031161000	大学物理B (二)	8	Spring	Compulsory	COMS0031161001
6	COEN0031161992	通用学术英语读写 (高级)	4	Autumn	Compulsory	
7	COMS0031132994	计算机动画	4	Spring	Elective	COMS0031132103
8	COMS0031132991	可信机器学习	4	Spring	Elective	
9	COEN0031111109	学术英语写作 (高级)	4	Spring	Compulsory	COEN0031161991, COEN0031161992
10	COMS0031132126	现代CAD技术 (B)	6	Spring	Elective	COMS0031132125
11	COMS0031132120	最优化方法	5	Spring	Elective	COMS0031131032
12	COMS0031132118	智能推荐系统	4	Spring	Elective	COMS0031121009, COMS0031132110
13	COEN0031161991	通用学术英语听说 (高级)	4	Autumn	Compulsory	
14	COMS0031132111	大数据系统	4	Spring	Elective	COMS0031132110
15	COMS0031132096	模式识别与机器学习	4	Spring	Elective	

2.生成排课:

(1)首先单击“设置约束”，在弹出的对话框输入每学期的学分上限以及禁排时段，设置完成后单击“OK”；

(2)接下来单击“生成排课”，在弹出的对话框输入最低总学分要求，设置完成后单击“OK”，即可生成选课方案；

(3)在课表上方点击不同学期以切换课表。



3.课程的“增、删、改、查”

(1)增：单击“添加课程”，依次输入课程 ID、课程名称、学分、开课学期、课程类型、先修课程和上课时间；



(2)删：切换到“课程列表”，选中想要删除的课程，单击“删除课程”即可删除；

(3)改：切换到“课程列表”，选中想要编辑的课程，单击“编辑课程”，依次输入课程名称、学分、开课学期、课程类型和先修课程；

(4)查：切换到“课程列表”，在右上角的对话框中输入课程 ID 或课程名称。



4.导出排课：单击“导出排课”即可生成 schedule.json 文件。

二、测试结果

导入 course.json。

1.基础功能之 300 学分以上选课方案测试：

智能选课系统

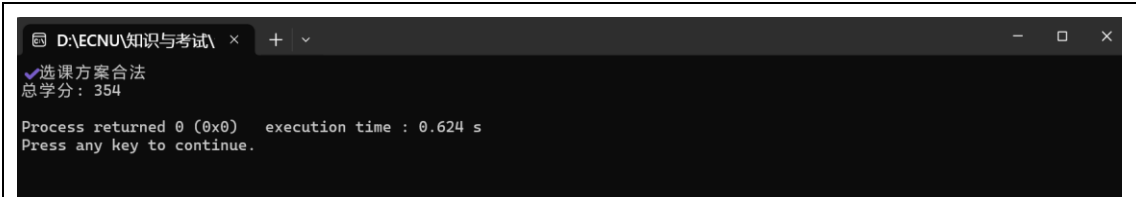
文件 帮助

导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 354 搜索课程...

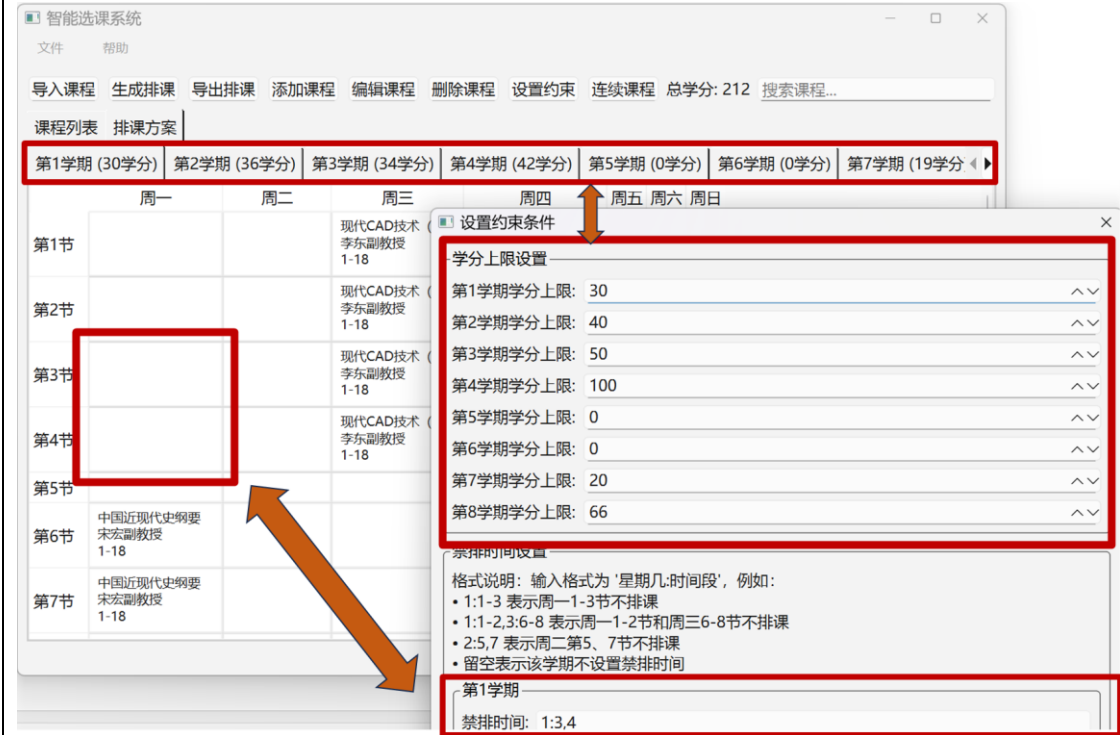
课程列表 排课方案

	第1学期 (52学分)	第2学期 (53学分)	第3学期 (49学分)	第4学期 (52学分)	第5学期 (28学分)	第6学期 (42学分)	第7学期 (46学分)
	周一	周二	周三	周四	周五	周六	周日
第1节		通用学术英语读写 (高级) 李辉 1-18	数学分析I 李智教授 1-18	程序设计原理与C语言 陆幼利副教授 1-18	通用学术英语听说 (高级) 李征 1-18		
第2节		通用学术英语读写 (高级) 李辉 1-18	数学分析I 李智教授 1-18	程序设计原理与C语言 陆幼利副教授 1-18	通用学术英语听说 (高级) 李征 1-18		
第3节	数学分析I 李智教授 1-18	计算机导论 周爱民教授 1-18		程序设计原理与C语言 陆幼利副教授 1-18	数学分析I 李智教授 1-18		
第4节	数学分析I 李智教授 1-18	计算机导论 周爱民教授 1-18		程序设计原理与C语言 陆幼利副教授 1-18	数学分析I 李智教授 1-18		
第5节							
第6节	中国近现代史纲要 宋宏副教授 1-18			大学物理B (一) 李波教授 1-18	计算机基础实践 金健副教授 1-18		

checker 评测结果：



经测试，接下来生成的所有方案均能通过 checker，限于篇幅不再展示 checker 评测截图。
2.扩展功能之学分上限与禁排时段测试：

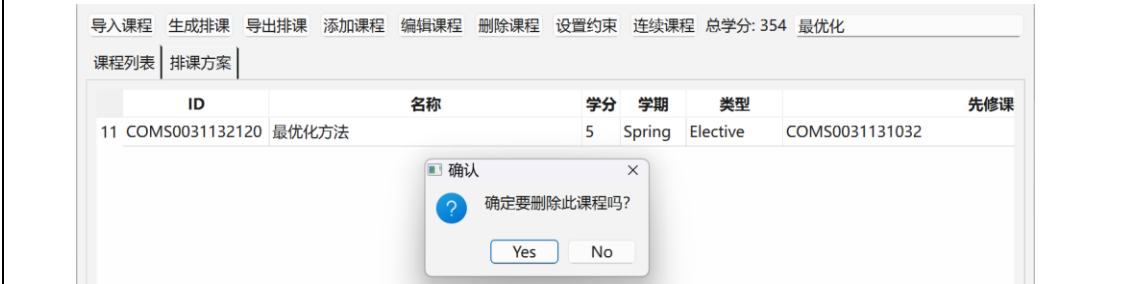


3.扩展功能之删除课程测试：

以“最优化方法（COMS0031132120）”为例，其上课时间为第 8 学期周五第 1~2 节。



执行删除操作



后第 8 学期周五第 1~2 节变为英语演说课（COEN0031111997）。

导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 349 最优化									
课程列表 排课方案									
学期 (53学分)	第3学期 (49学分)	第4学期 (52学分)	第5学期 (28学分)	第6学期 (42学分)	第7学期 (42学分)	第8学期 (31学分)			
周一	周二	周三	周四	周五	周六	周日			
第1节	网络安全基础 黄新力教授 1-18	计算机网络工程 陆刚副教授 1-18	英语演说课 王志宏 1-18						
第2节	网络安全基础 黄新力教授 1-18	计算机网络工程 陆刚副教授 1-18	英语演说课 王志宏 1-18						

4.扩展功能之查找课程、编辑课程测试：

查找“英语演说课”，其上课时间为第 8 学期周五第 1~2 节。

智能选课系统									
文件 帮助									
导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 349 英语									
课程列表 排课方案									
学期 (53学分)	第3学期 (49学分)	第4学期 (52学分)	第5学期 (28学分)	第6学期 (42学分)	第7学期 (42学分)	第8学期 (31学分)			
周一	周二	周三	周四	周五	周六	周日			
第1节	网络安全基础 黄新力教授 1-18	计算机网络工程 陆刚副教授 1-18	英语演说课 王志宏 1-18						
第2节	网络安全基础 黄新力教授 1-18	计算机网络工程 陆刚副教授 1-18	英语演说课 王志宏 1-18						

将其学分由 4 改为 8，开课学期由 Spring 改为 Autumn，

智能选课系统									
文件 帮助									
导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 349 英语									
课程列表 排课方案									
ID	名称	学分	学期	类型	先修课				
1 COEN0031112043	体育赛事英语	4	Spring	Elective	COEN0031111109				
5 COEN0031161992	通用学术英语读写 (高级)	4	Autumn	Compulsory					
8 COEN0031111109	学术英语写作 (高级)	4	Spring	Compulsory	COEN0031161991, COEN00311				
11 COEN0031161991	通用学术英语听说 (高级)	4	Autumn	Compulsory					
26 COEN0031111058	英语短篇小说赏析	4	Spring	Elective	COEN0031111109				
47 COEN0031111997	英语演说课	4	Spring	Elective	COEN0031111109				
65 COEN0031112046	英语畅谈中国文化	4	Autumn	Elective	COEN0031111109				

SmartCourseScheduler

课程名称: 英语演说课

学分: 8

开课学期: Autumn

课程类型: Elective

先修课程: COEN0031111109

OK Cancel

执行编辑操作后，上课时间变为第 3 学期周五第 1~2 节。



5. 扩展功能之添加课程测试

添加矩阵论 (COMS0031132138) 前, 第 2 学期周五第 1~2 节没有排课。



添加成功。



6. 扩展功能之连续课程设置。

数学分析 I (COMS0031121013) 在第 1 学期开课, 模式识别与机器学习 (COMS0031132096) 在第 4 学期开课,

导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 350 矩阵									
课程列表 排课方案									
第1学期 (52学分)		第2学期 (54学分)		第3学期 (49学分)		第4学期 (52学分)		第5学期 (28学分)	
周一		周二		周三		周四		周五	
第1节		通用学术英语读写 (高级) 李辉 1-18		数学分析I 李智教授 1-18		程序设计原理与C语言 陆幼利副教授 1-18		通用学术英语听说 (高级) 李征 1-18	
		通用学术英语读写 (高级) 李辉 1-18		数学分析I 李智教授 1-18		程序设计原理与C语言 陆幼利副教授 1-18		通用学术英语听说 (高级) 李征 1-18	
第3节	数学分析I 李智教授 1-18	计算机导论 周爱民教授 1-18				程序设计原理与C语言 陆幼利副教授 1-18		数学分析I 李智教授 1-18	
第4节	数学分析I 李智教授 1-18	计算机导论 周爱民教授 1-18				程序设计原理与C语言 陆幼利副教授 1-18		数学分析I 李智教授 1-18	

导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 350 矩阵									
课程列表 排课方案									
第1学期 (52学分)		第2学期 (54学分)		第3学期 (49学分)		第4学期 (52学分)		第5学期 (28学分)	
周一		周二		周三		周四		周五	
第4节	现代CAD技术 (B) 宋海川副教授 2-18 (双)	数学建模 石东昱讲师 1-18		编译原理与实践 谢瑾奎副教授 1-18				数据挖掘 兰曼教授 1-18	
				编译原理与实践 谢瑾奎副教授 1-18					
第6节	模式识别与机器学习 赵静副研究员 1-18	人工智能 周爱民教授 1-18				面向对象程序设计 (基于Java) 吴苑斌副教授 1-18		计算机组成与实践 金健副教授 1-18	
第7节	模式识别与机器学习 赵静副研究员 1-18	人工智能 周爱民教授 1-18				面向对象程序设计 (基于Java) 吴苑斌副教授 1-18		计算机组成与实践 金健副教授 1-18	

添加连续课程对

添加连续课程对

前导课程ID: COMS0031121013

后续课程ID: COMS0031132096

添加

当前连续课程关系

COMS0031121013 (数学分析I) → COMS0031132096 (模式识别与机器学习)

删除选中

Close

数学分析 I 仍在第 1 学期开课, 模式识别与机器学习开课时间变为第 2 学期。

智能选课系统									
文件 帮助									
导入课程 生成排课 导出排课 添加课程 编辑课程 删除课程 设置约束 连续课程 总学分: 317 矩阵									
课程列表 排课方案									
第1学期 (52学分)		第2学期 (54学分)		第3学期 (40学分)		第4学期 (43学分)		第5学期 (25学分)	
周一		周二		周三		周四		周五	
第4节	数学分析II 沈超敏副教授 1-18			线性代数 王峰副研究员 1-18		大学物理B (二) 李波教授 1-18		数学分析II 沈超敏副教授 1-18	
第6节	模式识别与机器学习 赵静副研究员 1-18	大学物理B (二) 李波教授 1-18				多媒体技术 杨静副教授 1-18		统计学习算法导论 张志忠 2-18 (双)	
第7节	模式识别与机器学习 赵静副研究员 1-18	大学物理B (二) 李波教授 1-18				多媒体技术 杨静副教授 1-18		统计学习算法导论 张志忠 2-18 (双)	

LLM 使用情况

开发者对于 Qt 交互界面开发并不熟悉，首先使用了 DeepSeek 生成.ui 文件。

提示词：“我需要 ai 帮我设计一个美观的智能选课系统界面，要求界面可视化与现代化，并且清晰简洁，需要设计的框架包含课程列表可视化、8 学期课表可视化、约束设置、增删改查等多个功能模块的界面。技术需求是包含课程列表、8 学期课表、约束设置等多个功能模块的界面。设计方面，要求界面要美观、布局合理、操作便捷。”

结果：见 mainwindow.ui。

接下来，开发者在排课算法中陷入了死循环：

// 在 fillSemesterToMaximum()中观察到的危险循环

```
while (availableSlots > 0) { // 可能无法退出！
```

```
    bool addedCourse = false;
```

```
    for (const Course &course : remainingCourses) {
```

```
        if (/* 满足条件 */) {
```

```
            addedCourse = true;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (!addedCourse) break; // 依赖此条件退出
```

```
}
```

经过 cursor 的指导，我增加了逃脱机制：

```
int maxAttempts = 100; // 安全计数器
```

```
while (availableSlots > 0 && maxAttempts-- > 0) {
```

```
    // ...原有逻辑...
```

```
}
```

```
if (maxAttempts <= 0) qWarning() << "排课可能未完成";
```

和精确计算可用时间槽：

```
availableSlots = calculateAvailableTimeSlots(semester); // 每次循环后重新计算
```

小组成员分工

符浩峰：核心排课功能、添加课程功能、限制学分上限、录制视频；

王业涵：删除课程功能、编辑课程功能、修改课程功能、撰写报告；

协作部分：设计并实现 Qt 用户界面、开发交互功能、共同讨论需求和系统设计。

反思

一、完成项目的感想

通过本次项目开发，我们深刻体会到了理论与实践相结合的重要性。在课堂上学习的 Qt 框架、C++ 编程、算法设计等知识，在实际项目中得到了充分的应用和验证。作为两人小组，我们深刻体会到了分工协作的重要性，通过定期的代码评审和讨论，我们不仅提高了代码质量，也加深了对彼此工作内容的理解，形成了良好的协作机制。当看到智能选课系统最终能够自动生成满足各种约束条件的排课方案，并且提供友好的用户界面和交互体验时，这种从需求分析到功能实现的完整过程，让我们体会到了软件开发的魅力和成就感。

二、对课程的意见和建议

该课程将理论与实践结合，不仅教授理论知识，更注重实际项目开发，让我们能够将所学知识应用到具体项目中。建议教师提供 Qt 可视化界面的操作方法，帮助学生熟悉.ui 组件的使用。

参考文献

[1]全面掌握自动排课算法与软件开发:

https://blog.csdn.net/weixin_34520664/article/details/142101820

[2]Qt 官方文档. Qt 6.0 Documentation[EB/OL]: <https://doc.qt.io/qt-6/>, 2023.

[3]高校选课系统设计与实现: https://blog.csdn.net/qc_1406299528/article/details/134476063

大型语言模型使用情况： 请在此部分描述在项目过程中如何使用大模型进行辅助（如有）

小组分工及每个人的具体工作： 姓名 1： 姓名 2（如有）：
完成项目的感想、对课程的意见和建议：

参考文献（如有，请列出项目过程中参考的具体文献，包括论文、博客、公众号等）